西安電子科技大學
XIDIAN UNIVERSITY

Feature Selection

Machine Learning (Bilingual)

Li Yang
(liy@xidian.edu.cn)

School of Artificial Intelligence

Xidian University

Contents

Subset search and evaluation

Subset search

Feature selection

- There are many properties to describe an object, which are called features in machine learning
- Properties useful for the learning task are called **relevant features**, while not so useful properties are called **irrelevant features**
- The process to choose a subset of relevant features is called **feature selection**

Feature selection

- Why we need feature selection?
  - Curse of dimensionality
  - To leverage the difficulty of a learning task
- Feature selection should not discard important features
  - Irrelevant features refer to features irrelevant to the current learning task.
  - **Redundant features** can be deduced from other features.
  - Here we assume that there are no redundant features.

Feature Selection

- The first step in feature selection is to define a criterion function that is often a function of the classification error.
- Note that, the use of classification error in the criterion function makes feature selection procedures dependent on the specific classifier used.
- The most straightforward approach would require
  - examining all $\binom{d}{m}$ possible subsets of size m,
  - selecting the subset that performs the best according to the criterion function.
- The number of subsets grows combinatorially, making the exhaustive search impractical.
- Iterative procedures are often used but they cannot guarantee the selection of the optimal subset.

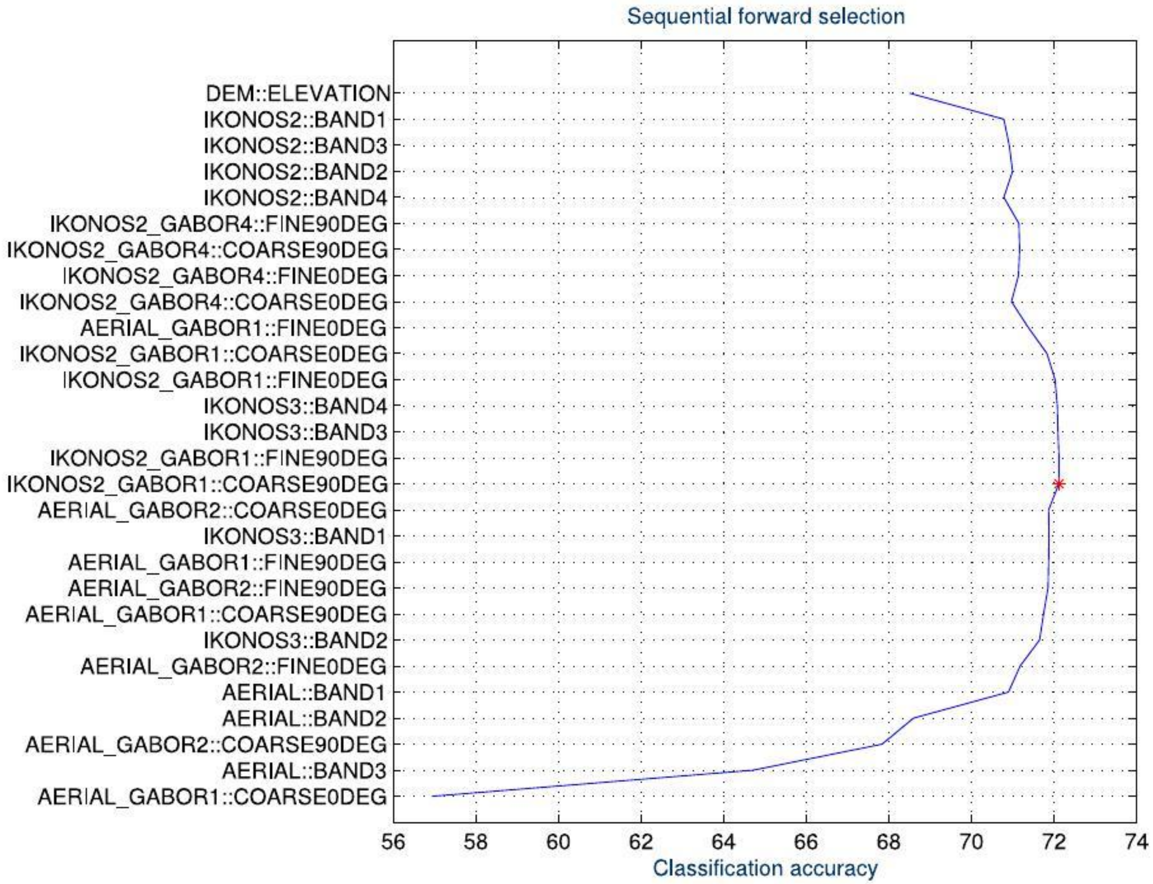
Sequencial subset search

- First, the best single feature is selected.
- Then, pairs of features are formed using one of the remaining features and this best feature, and the best pair is selected.
- Next, triplets of features are formed using one of the remaining features and these two best features, and the best triplet is selected.
- This procedure continues until all or a predefined number of features are selected.
  - Assume that in the $(k+1)$-th round, the best candidate subset is no better than the previous round, then stop.
- As a greedy strategy, no optimality is guaranteed.

Subset search

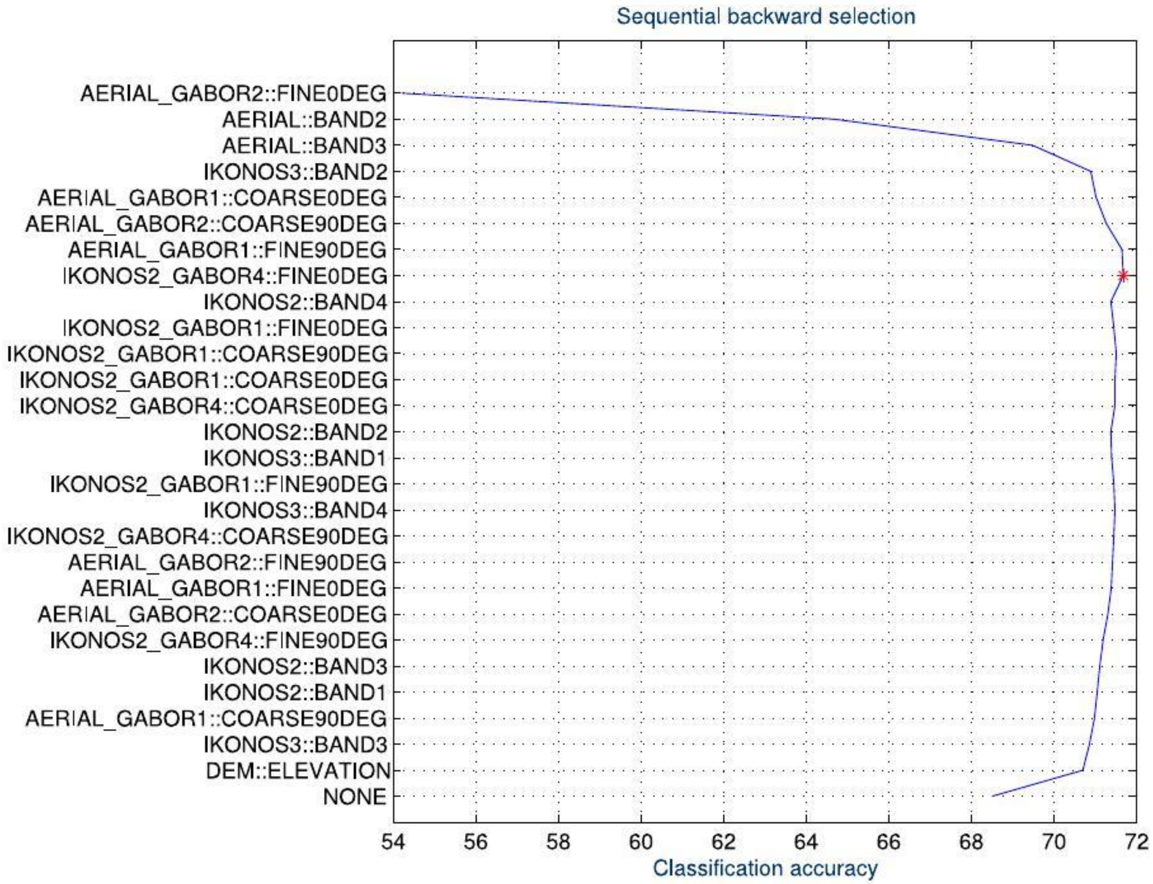
- The strategy of incrementally adding features is called “forward search”, and is known as “sequential forward selection”.
- We can also begin with the whole set of features, and remove one feature at each iteration, which is called “sequential backward selection”.
- In addition, a strategy combining both forward and backward selection can be applied, which increases relevant feature and decreases irrelevant feature, and is called “bidirectional search”.

Examples



Results of sequential forward feature selection for classification of a satellite image using 28 features. x-axis shows the classification accuracy (%) and y-axis shows the features added at each iteration (the first iteration is at the bottom). The highest accuracy value is shown with a star.

Examples



Results of sequential backward feature selection for classification of a satellite image using 28 features. x-axis shows the classification accuracy (%) and y-axis shows the features removed at each iteration (the first iteration is at the bottom). The highest accuracy value is shown with a star.

Method	Property
Exhaustive Search	Evaluate all $\binom{d}{m}$ possible subsets.
Branch-n-Bound Search	Uses the well-known branch-and-bound search method; only a fraction of all possible feature subsets need to be enumerated to find the optimal subset.
Best Individual Feature	Evaluate all the m features individually; select the best m individual features

Method	Property
Sequencial Forward Selection (SFS)	Select the best single feature and then add one feature at a time which in combination with the selected features maximizes the criterion function.
Sequencial Backward Selection (SBS)	Start with all the d features and successive delete one feature at a time.
"Plus l-take away r" Selection	First enlarge the feature subset by l features using forward selection and then delete r features using backward selection.
Sequencial Forward Floating Search (FSFS) and Sequencial Backward Floating Search (SBFS)	A generalization of "plus l-take away r" method; the values of l and r are determined automatically and updated dynamically.

Subset evaluation

Subset evaluation

- For a given data set $\mathcal{D}$, assume that the ratio of samples the i-th class is p_i.
- Assume that properties are discrete.
- For subset A of properties, data set $\mathcal{D}$ can be divided into V subsets, $\{\mathcal{D}^1, \mathcal{D}^2, \ldots, \mathcal{D}^V\}$ according to the value of features.
- Then for property subset A, the information gain is calculated as

$$\mathrm{Gain}(A) = \mathrm{Ent}(\mathcal{D}) - \sum_{v=1}^{V} \frac{|\mathcal{D}^v|}{|\mathcal{D}|} \mathrm{Ent}(\mathcal{D}^v)$$

where the entropy is defined as

$$\mathrm{Ent}(\mathcal{D}) = -\sum\nolimits_{n=1}^{N} p_n \log_2 p_n.$$

Subset search and evaluation

- Combining mechanisms of subset search and subset evaluation, we can establish feature selection algorithms.
- Feature selection algorithms explicitly or implicitly combine certain or multiple subset search and evaluation mechanisms.
  - Filter
  - Wrapper
- Sparse representation and ℓ_1 regularization

Filter

- Within filtering approach, features are selected before training the learning algorithm, where feature selection does not related to the training procedure.
- **Relief**, or relevant features (Kira and Rendell, 1992) designs a "relevant statistics" to measure the importance of features.
- The statistics is a vector, where each element corresponds to a feature
- The importance of a feature subset is measured by the sum of all relevant statistics.
  - So we can use a threshold τ to choose features with a statistics larger than τ;
  - or given the number of features k to select the k features with maximum statistics.

Filter

- The key of Relief is the relevant statistics
- For a sample $\mathbf{x}_i$,
  - get the nearest neighbor $\mathbf{x}_{i,\mathrm{nh}}$ within the same class, called **near-hit**,
  - and get the nearest neighbor $\mathbf{x}_{i,\mathrm{nm}}$ that is not within the same class, called **near-miss**.
- Then the relevant statistics for feature j is given by

$$\delta^j = \sum_i -\mathrm{diff}(\mathbf{x}_i^j, \mathbf{x}_{i,\mathrm{nh}}^j)^2 + \mathrm{diff}(\mathbf{x}_i^j, \mathbf{x}_{i,\mathrm{nm}}^j)^2,$$

- If the sample is near to the near-hit neighbor than the near-miss one, the feature is useful for discriminant of the class label
- otherwise, the effect is negative.
- Averaging over all samples gives the discriminant capability of each feature.

Wrapper

Wrapper

- Different to filtering, wrapper employs the performance of the learner as the measurement of subsets.
- From the perspective of performance of the learner, wrapper is usually better than filter.
- However, because the multiple training of the learner, wrapper based feature selection has a computational cost far larger than filter.

Las Vegas Wrapper

- Las Vegas Wrapper, or LVW (Liu and Setiono, 1996) is a typical wrapper based on random strategy within the Las Vegas framework.
- Note: Las Vegas vs Monte Carlo

- **Input**: Data set $\mathcal{D}$, feature set A, learner L, stop time T
- **Procedure**
- $E = \infty, d = |A|, A^* = A, t = 0$
- **while** $t < T$ **do**
  - Generate random subset A';
  - $d' = |A'|$;
  - $E' = \mathrm{CrossValidation}(L(\mathcal{D}^{A'}))$;
  - **if** $(E' < E) \vee ((E' = E) \wedge (d' < d))$ **then**
    - $t = 0,\ E = E',\ d = d',\ A* = A'$;
  - **else**
    - $t = t + 1$;
  - **endif**
- **end while**

西安电子科技大学
XIDIAN UNIVERSITY

Sparse Representation

Embedding with ℓ_1 Regularization

Embedded feature selection

- In filters and wrappers, feature selection are separated from training;
- Different to them, embedded feature selection combines both procedures of feature selection and learner training as a whole, and accomplished in the same optimization.
- In the training procedure of the learner, feature selection has been conducted automatically.

Embedded feature selection

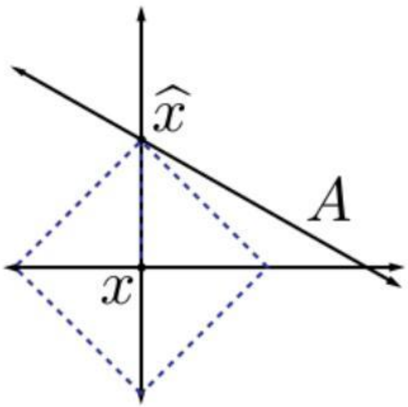
- Recall the regularized linear regression

$$J(\mathbf{w}) = \sum_{i=1}^{m} \left(y^{(i)} - y(\mathbf{x}^{(i)}, \mathbf{w}) \right)^2 + \lambda \|\mathbf{w}\|_2^2$$

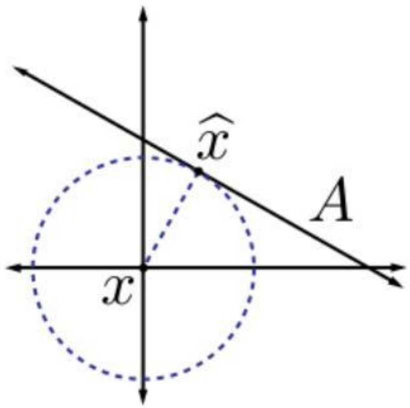
- Replacing the ℓ_2-norm with the ℓ_1-norm, we have LASSO (Least Absolute Shrinkage and Selection Operator):

$$J(\mathbf{w}) = \sum_{i=1}^{m} \left(y^{(i)} - y(\mathbf{x}^{(i)}, \mathbf{w}) \right)^2 + \lambda \|\mathbf{w}\|_1$$

Embedded feature selection



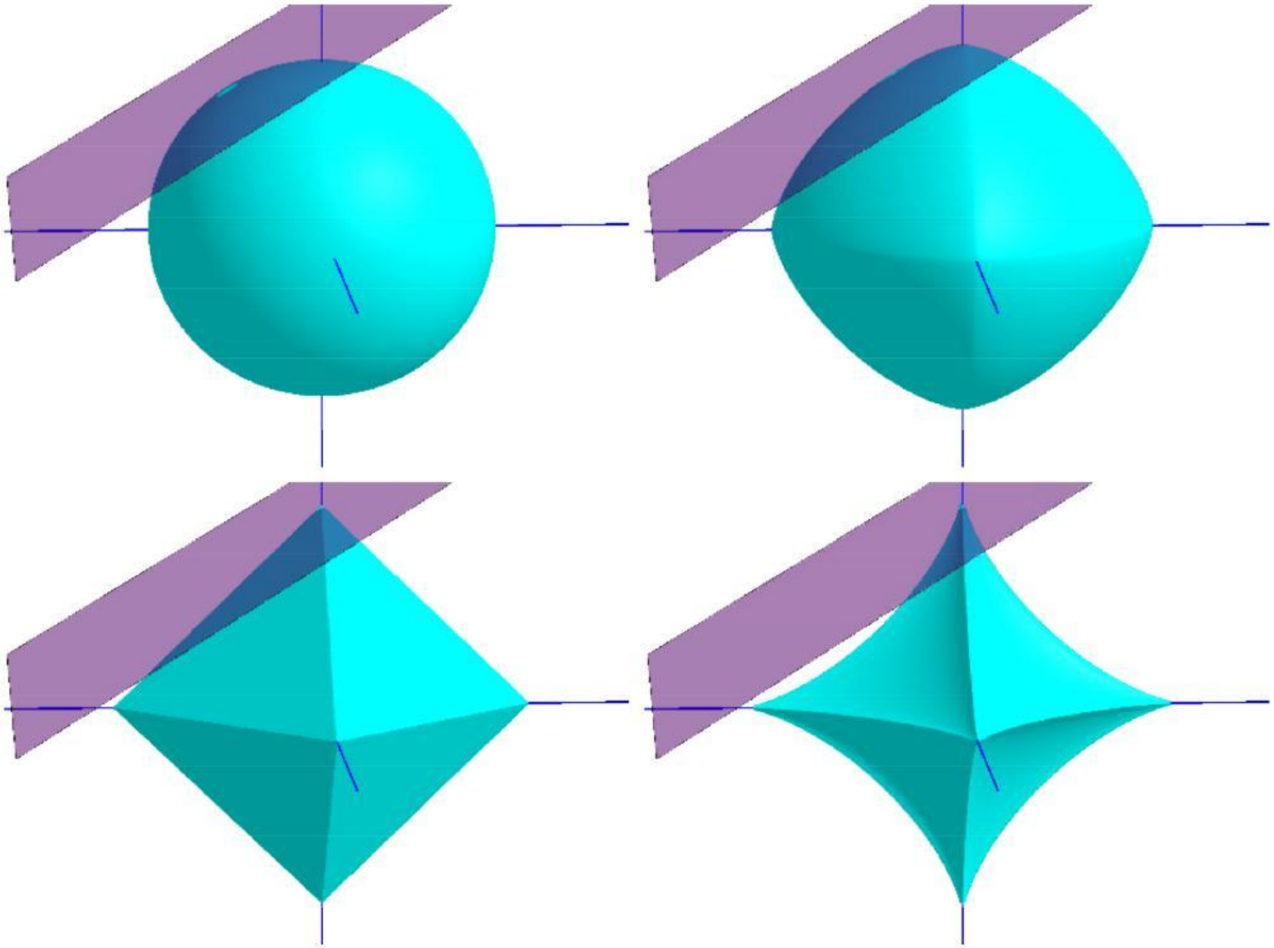
$p = 1$



$p = 2$

- The advantage of employing the ℓ_1-norm is that it tends to obtain **sparse** solutions, i.e., there will be fewer non-zero entries in $\mathbf{w}$.
- The ℓ_1 regularization can be solved via proximal gradient descent, or PGD (Boyd and Vandenberghe, 2004).

Embedded feature selection



Intersections demonstrated for $p = 2, 1.5, 1, 0.7$. Figure from: Elad, M. *Sparse and Redundant Representations*. (Springer, 2010).

- The intersection between the ℓ_p-ball and the set $Ax = b$ defines the solution. When $p \leq 1$, the intersection takes place at a **corner** of the ball, leading to a **sparse** solution.

Dictionary Learning

Dictionary Learning

- If we treat the data set $\mathcal{D}$ as a matrix, each row is a sample while each column corresponds to a feature.
- In feature selection, features are considered be with "**sparsity**", i.e., many columns are uncorrelated to the learning task.
- Now let us consider another type of sparsity: the matrix of $\mathcal{D}$ have many zeros but are not regular rows or columns.
- We need to find an adequate dictionary that samples are transformed to some **sparse representation** such that the learning task can be simplified or the model complexity is reduced.
- This is usually called **dictionary learning** or **sparse coding**.

- For a given data set $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m\}$, the simplest form of dictionary learning is

$$\min_{\mathbf{B}, \boldsymbol{\alpha}_i} \sum_{i=1}^{m} \|\mathbf{x}_i - \mathbf{B}\boldsymbol{\alpha}_i\|_2^2 + \lambda \sum_{i=1}^{m} \|\boldsymbol{\alpha}_i\|_1, \qquad \text{(SR)}$$

  - where $\mathbf{B} \in \mathbb{R}^{d \times k}$ is the dictionary matrix,
  - k is the vocabulary of the dictionary,
  - and $\boldsymbol{\alpha}_i \in \mathbb{R}^k$ is the sparse representation of its corresponding sample $\mathbf{x}_i \in \mathbb{R}^d$.
- In (SR), the first term expects better reconstruction while the second term constraints $\boldsymbol{\alpha}_i$ to be sparse enough.

Dictionary Learning

- The optimization of (SR) can be solved via alternating iteration.
- In the first stage, find $\boldsymbol{\alpha}_i$ for each sample $\mathbf{x}_i$ with a strategy similar to LASSO,

$$\min_{\boldsymbol{\alpha}_i} \|\mathbf{x}_i - \mathbf{B}\boldsymbol{\alpha}_i\|_2^2 + \lambda\|\boldsymbol{\alpha}_i\|_1.$$

- In the second stage, we fix $\boldsymbol{\alpha}_i$ to update the dictionary,

$$\min_{\mathbf{B}} \|\mathbf{X} - \mathbf{B}\mathbf{A}\|_F^2,$$

  - where $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_m) \in \mathbb{R}^{d\times m}$, $\mathbf{A} = (\boldsymbol{\alpha}_1, \boldsymbol{\alpha}_2, \ldots, \boldsymbol{\alpha}_m) \in \mathbb{R}^{k\times m}$,
  - and $\|\cdot\|_F$ denotes the Frobenius norm of a matrix.
- The optimal solution the second stage can be obtained using the column-wise updating strategy like KSVD (Aharon *et al.*, 2006).